

COMICCRAFT – AI COMIC STORY CREATOR

Project Development Phase

Project Type: Web-based AI Comic Story Generator
Development Stack: FastAPI, Google Gemini Models, Stable Diffusion, Hugging Face Diffusers, Jinja2, PyTorch and FPDF

1. Development Overview

The development phase implements the planned ComicCraft workflow as a web application. The system receives comic preferences from the user, generates a structured story outline, creates narration and dialogue, generates panel illustrations, assembles the content into a comic layout, and exports the result as a PDF.

2. Development Environment

- Python is used as the primary development language.
- FastAPI provides the web backend and request handling.
- Uvicorn runs the FastAPI application locally.
- HTML, CSS and Jinja2 are used for the web interface and dynamic templates.
- Google Gemini APIs provide story-generation capabilities.
- Hugging Face Diffusers with Stable Diffusion provides image generation.
- PyTorch supports the image-generation workflow.
- FPDF is used to create the final comic PDF.
- Git and virtualenv are included in the project prerequisites.

3. AI Model Development

AI Component	Development Responsibility
Gemini Flash	Generate a structured five-panel comic outline with panel number, title, scene description and image p
Gemini Pro	Generate detailed narration and dialogue for the comic story.
Stable Diffusion	Generate comic-style illustrations based on the panel image prompts.

4. Core Function Development

4.1 Outline Generation – gemini_flash.py

The outline-generation component implements **generate_outline()**. It produces a five-panel outline and provides the structured information needed by later development stages, including panel number, panel title, scene description and image prompt.

4.2 Story Generation – gemini_pro.py

The story-generation component implements **generate_story()**. It uses the generated outline and project inputs to produce narration and dialogue for the comic panels.

4.3 Image Generation – `image_generator.py`

The image-generation component implements `generate_image()`. Stable Diffusion is used to create comic-style panel illustrations, which are saved under the project's `static/panels` location.

4.4 Comic Layout – `layout_builder.py`

The layout component implements `build_comic_layout()`. It combines the generated story information and panel images so that each panel contains the corresponding visual and textual content.

4.5 PDF Export – `exporters.py`

The export component implements `save_pdf()` using FPDF. The completed comic is assembled into a multi-page PDF and stored in `static/exports`.

5. FastAPI Backend Development

The FastAPI backend connects the frontend to the AI and export components. The routing layer receives user input, invokes the generation functions in sequence, prepares the comic layout, and returns the generated result to the appropriate template or JSON response.

Route	Development Purpose
/	Display the ComicCraft homepage and input form.
/generate	Receive form input and run the complete comic-generation workflow.
/generate-comic/json	Accept JSON input and return generated layout/PDF information.
/export-success	Display the successful export confirmation page.
/test-image	Provide a direct image-generation test route.

6. End-to-End Development Workflow

1. Receive the user's story prompt and comic preferences.
2. Process the request through the FastAPI route.
3. Generate a five-panel outline with Gemini Flash.
4. Generate narration and dialogue with Gemini Pro.
5. Generate panel illustrations using Stable Diffusion.
6. Save generated panel images in the static/panels location.
7. Match the generated images with their corresponding story content.
8. Build the comic preview using the Jinja2 template.
9. Assemble the comic into a multi-page PDF using FPDF.
10. Store the PDF in static/exports and show the export-success page.

7. Frontend Development

- **index.html:** Provides fields for story prompt, character name, setting, tone and art style.
- **comic_preview.html:** Displays panel title, image, scene description, captions and narration.
- **export_success.html:** Confirms that the comic PDF has been exported and provides a way to create another comic.
- Jinja2Templates is used to pass generated AI results dynamically into the HTML pages.
- The interface is designed to be clean and responsive for the comic-generation workflow.

8. Environment and Configuration Development

The project uses a Python virtual environment and installs the required application and AI dependencies. API credentials are configured through environment variables, including **GEMINI_API_KEY** and **HF_API_KEY**. The FastAPI application is run with Uvicorn.

Local development command specified in the project:

```
uvicorn app.main:app --reload
```

The application can be accessed through the local server, with the FastAPI documentation available through the **/docs** endpoint.

9. Comic Panel Development

Panel Element	Development Role
Panel number	Maintains the sequence of the comic.
Panel title	Provides the panel heading.
Scene description	Describes the action or visual scene.
Image prompt	Provides the description used for illustration generation.
Illustration	Generated visual corresponding to the panel.
Caption / narration	Adds the generated story text to the panel.
Dialogue	Provides character speech where generated.

10. Error Handling and Integration

The routes layer includes error handling around the AI-generation workflow. The development process connects the individual model functions so that outline generation, story generation, image creation, layout assembly and PDF export operate as one application flow.

11. Development Verification Points

- Verify that the homepage accepts the required story inputs.
- Verify that Gemini Flash produces the expected five-panel outline structure.
- Verify that Gemini Pro produces narration and dialogue.
- Verify that Stable Diffusion generates and saves panel images.
- Verify that the generated images correspond to the correct panels.
- Verify that the comic preview renders generated content through Jinja2.
- Verify that the PDF export contains the generated images and text.
- Verify the export-success page after PDF creation.
- Verify the /docs API documentation endpoint during local development.

12. Development Deliverables

Deliverable	Result
AI generation modules	Outline, narration/dialogue and image generation functionality
FastAPI backend	Routes and orchestration of the comic-generation process
Frontend templates	Input, comic preview and export-success interfaces
Panel layout	Combined text and image comic structure
PDF exporter	Downloadable multi-page comic
Local application	Runnable ComicCraft web application

13. Development Outcome

The development phase implements the planned ComicCraft architecture into an integrated application workflow. The resulting system connects user input, Gemini-based story generation, Stable Diffusion illustration generation, panel assembly, browser preview and FPDF-based comic export into a single process.

The project document describes the current version as a single-comic generation system, with future possibilities including user profiles, comic libraries and multi-page story arcs.

Source basis: ComicCraft – AI Comic Story Creator using Gemini Models project document.